

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

1 • Ön-öğretim **Ön Bilgi:** Hafta 2'den taşıdıklarınız: `const` doğruluğu ve referans (`const T&` — kopyasız salt-okunur geçiş), namespace, `.hpp/.cpp` ayrımı ve CMake iskeleti, yığın/öbek ömrü ayrımı, `new/delete` ve ilk RAII örneği (kurucu/yıkıcı ile sınımlı `IntTampon`). C'den tanıdık gelen: struct tanımı ve serbest fonksiyon (yalnız karşılaştırma noktası — verisi korumasız, davranışı kopuk). Bu hafta aynı veriyi ve davranışı erişim kontrolüyle tek çatı altında toplayan sınıfı kurar; kurucu/yıkıcı, üye başlatma listesi ve `this` ile nesneyi ilk andan tutarlı biçimde oluşturunuz.

» Sınıf ve Erişim

Sınıf Tanımı

Veri (alanlar) ve davranışı (üye fonksiyonlar) bir arada tutan kullanıcı tipi.

```
1 class Kitap {
2 public:
3     Kitap(std::string b, int y);
4     void yazdir() const;
5 private:
6     std::string baslik_;
7     int yil_;
8 };
```

Erişim Belirleyiciler

public: sınıf dışından erişilir. **private:** yalnız sınıf içinden. Varsayılan: `struct`'ta **public**, `class`'ta **private**.

Kapsülleme (Encapsulation)

Private veri + public arayüz: alanlara yalnız doğrulamalı üye fonksiyonlar (getter/setter) üzerinden erişilir, dışarıdan doğrudan bozulaamaz.

» Kurucu, this, Yıkıcı

Kurucu (Constructor)

Sınıfla aynı ad, dönüş tipi yok; nesne oluşurken otomatik çağrılır. Parametrelili kurucu ilk durumu kurar; hiç kurucu yazılmazsa derleyici varsayılan (boş) kurucuyu oluşturur, ama bu kurucu alanları ilklemez.

Üye Başlatma Listesi

: **alan_(deger)** — gövdede atamaya tercih edilir (tek adımda ilkler, önce-boşalt-sonra-doldur israfı yoktur); **const/referans** üye için ZORUNLUDUR.

```
1 Kitap::Kitap(std::string b, int y)
2 : baslik_(std::move(b)), yil_(y) {}
```

this İşaretçisi

Çağrıldığı nesneyi gösteren örtük işaretçi; **this->uye** ile parametre-üye isim çakışması çözülür.

const Üye Fonksiyon

tip fn() const — nesnenin durumunu değiştirmez (getter); gövdede üyeye yazma DERLEME HATASI verir. Kurucu/setter **const** DEĞİLDİR.

Yıkıcı (Destructor)

~Ad() — parametresiz, nesnenin ömrü bitince (kapsam sonu/delete) otomatik çağrılır; kaynak serbest bırakma burada yapılır.

» Kitap Sınıfı: Tam İskelet

Private Alanlar + Public Arayüz

```
1 namespace kitaplik {
2 class Kitap {
3 public:
4     Kitap(std::string baslik, std::string yazar,
5           std::string isbn, int yil);
6     const std::string& baslik() const;
7     void setYil(int yil);
8     bool gecerliMi() const;
9     void yazdir() const;
10 private:
11     std::string baslik_, yazar_, isbn_;
12     int yil_;
13 };
14 } // namespace kitaplik
```

Beş bildirim = **Kitap** ile dışarıdan kurulabilecek her işi: kurucu, getter, setter, doğrulama, yazdırma.

Getter/Setter Deseni

```
1 const std::string& baslik() const { return baslik_; }
2 void setYil(int yil) { if (yil > 0) yil_ = yil; }
```

Getter: **const**, referansla döner, kopyalamaz, değiştirmez. Setter: parametreyi doğrulayıp yalnız geçerliyse yazar — kural sınıfın içinde kalır, dışarıdaki koda güvenilmez.

Karmaşıklık (big-O)

Bir getter/setter tek bir alana doğrudan erişir: alanın nesne içindeki ofseti derleme anında bellidir, arama/döngü gerekmez. **baslik()** ya da **setYil()** çağrısı $O(1)$ — nesnenin kaç alan taşıdığından bağımsızdır.

» Sık Yapılan Hatalar ve Seçimler

Ortak Hata / Tuzak

- Gövdede atama (**baslik_ = b;**) init listesi yerine: üye önce varsayılan kurulur, sonra üzerine yazılır — gereksiz çift işlem.
- const/referans** üyeye gövdede atama: DERLEME HATASI; yalnız init listesinde ilklenebilir.
- yazar_/isbn_** gibi bir alanı ne listede ne gövdede ilklemeyi unutmak: alan tanımsız/çöp değerde sessizce kalır.
- struct** içinde varsayılan erişim **public**'tir — bölüm etiketi yazmayı unutursanız alan açıkta kalır, kapsülleme kaybolur.

Ne Zaman Hangisi?

- Init listesi vs gövde atama: **const/referans** üye varsa liste ZORUNLU; diğer üyelerde de liste tercih edilir (tek adım, verimli).
- struct** vs **class**: dışarıdan serbest erişilen sade veri paketiye **struct**; doğrulama/kapsülleme gerekiyorsa **class**.
- Getter neden **const**: nesneyi değiştirmedigini derleyiciye garanti eder; böylece **const Kitap&** üzerinde de çağrılabilir.

» Kitaplığı Kurmak

Başlık/Kaynak Ayrımı

.hpp: sınıf bildirimi, başında `#pragma once` (çoklu include koruması — aynı başlık iki kez yapıştırılırsa “yeniden tanımlama” hatası oluşur). .cpp: üye fonksiyon tanımları. Katalog (.hpp) vs raf (.cpp); .cpp değişikliği ucuz (yalnız o dosya derlenir), .hpp değişikliği pahalıdır (onu include eden her dosya yeniden derlenir).

namespace

`namespace kitaplik { class Kitap { ... }; }` — isim çakışmasını önler; *Kitap*, dönem boyu büyüyecek kütüphanenin ilk sınıfıdır.

```
g++ -std=c++20 -c kitap.cpp
```

Kaynağı nesne dosyasına derler

```
g++ kitap.o kitap_demo.o -o demo
```

Nesne dosyalarını bağlar

```
cmake -B build && cmake --build build
```

CMake ile yapılandırıp derler

5 • İzleme

Nerede Kullanılır: Bir alanı dışarıdan gelen tutarsız değere karşı korumak istediğinizde onu `private` yapıp yalnız doğrulayan bir arayüzden (`setYıl()` gibi) eriştirin; salt okunacak alanı `const` referansla döndüren `const` getter ile açın. Nesneyi ilk andan tutarlı kurmak için parametrelili kurucu + üye başlatma listesi kullanın — `const`/referans üyede bu ZORUNLUDur. Bir kuralın (doğrulama) unutulmadan her yerde geçerli olmasını istiyorsanız onu serbest fonksiyona değil sınıfın içine koyun. Kütüphane büyüyüp yeniden derleme maliyeti arttığında sınıfı .hpp (bildirim) / .cpp (tanım) olarak ayırıp namespace ile çakışmayı önleyin. Bu desen (*Kitap*) dönem boyu eklenecek her yeni sınıfta aynen tekrarlanır.